

RSA cryptosystem

The **RSA (Rivest–Shamir–Adleman) cryptosystem** is a family of public-key cryptosystems (one of the oldest), widely used for secure data transmission. The initialism "RSA" comes from the surnames of Ron Rivest, Adi Shamir and Leonard Adleman, who publicly described the algorithm in 1977.^{[1][2][3]} An equivalent system was developed secretly in 1973 at Government Communications Headquarters (GCHQ), the British signals intelligence agency, by the English mathematician Clifford Cocks. That system was declassified in 1997.^[4]

RSA is used in digital signature such as RSASSA-PSS or RSA-FDH,^{[5][6][7][8][9][10]} public-key encryption of very short messages (almost always a single-use symmetric key in a hybrid cryptosystem) such as RSAES-OAEP,^{[11][12][13][10]} and public-key key encapsulation.^{[14][15][16]}

In RSA-based cryptography, a user's *private key*—which can be used to sign messages, or decrypt messages sent to that user—is a pair of large prime numbers chosen at random and kept secret. A user's *public key*—which can be used to verify messages from the user, or encrypt messages so that only that user can decrypt them—is the product of the prime numbers.

The security of RSA is related to the difficulty of factoring the product of two large prime numbers, the "factoring problem". Breaking RSA encryption is known as the RSA problem. Whether it is as difficult as the factoring problem is an open question.^[17] There are no published methods to defeat the system if a large enough key is used.

History

The idea of an asymmetric public-private key cryptosystem is attributed to Whitfield Diffie and Martin Hellman, who published this concept in 1976. They also introduced digital signatures and attempted to apply number theory. Their formulation used a shared-secret-key created from exponentiation of some number, modulo a prime number. However, they left open the problem of realizing a one-way function, possibly because the difficulty of factoring was not well-studied at the time.^[18] Moreover, like Diffie-Hellman, RSA is based on modular exponentiation.

Ron Rivest, Adi Shamir, and Leonard Adleman at the Massachusetts Institute of Technology made several attempts over the course of a year to create a function that was hard to invert. Rivest and Shamir, as computer scientists, proposed many potential functions, while Adleman, a mathematician, was responsible for finding their weaknesses. They tried many approaches,

RSA cryptosystem

General

Designers Ron Rivest, Adi Shamir, and Leonard Adleman

First published 1977

Certification PKCS#1, ANSI X9.31

Cipher detail

Key sizes variable but typically 2,048 to 4,096 bits

Rounds 1

Best public cryptanalysis

General number field sieve for classical computers;

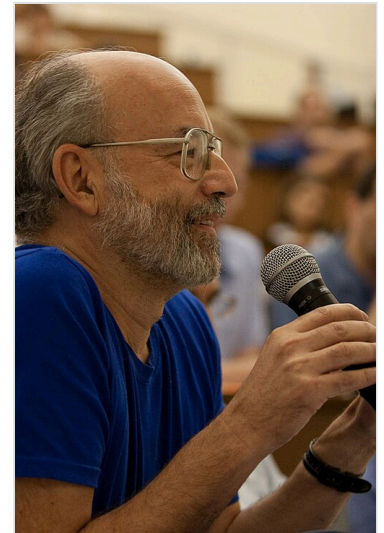
Shor's algorithm for quantum computers.

An 829-bit key has been broken.

including "knapsack-based" and "permutation polynomials". For a time, they thought what they wanted to achieve was impossible due to contradictory requirements.^[19] In April 1977, they spent Passover at the house of a student and drank a good deal of wine before returning to their homes at around midnight.^[20] Rivest, unable to sleep, lay on the couch with a math textbook and started thinking about their one-way function. He spent the rest of the night formalizing his idea, and he had much of the paper ready by daybreak. The algorithm is now known as RSA – the initials of their surnames in same order as their paper.^[21]

Clifford Cocks, an English mathematician working for the British intelligence agency Government Communications Headquarters (GCHQ), described a similar system in an internal document in 1973.^[22] However, given the relatively expensive computers needed to implement it at the time, it was considered to be mostly a curiosity and, as far as is publicly known, was never deployed. His ideas and concepts were not revealed until 1997 due to its top-secret classification.

Kid-RSA (KRSA) is a simplified, insecure public-key cipher published in 1997, designed for educational purposes. Kid-RSA gives insight into RSA and other public-key ciphers, analogous to simplified DES.^{[23][24][25][26][27]}



Adi Shamir, co-inventor of RSA (the others are Ron Rivest and Leonard Adleman)

Patent

A patent describing the RSA algorithm was granted to MIT on 20 September 1983: U.S. patent 4,405,829 (<https://patents.google.com/patent/US4405829>) "Cryptographic communications system and method". From DWPI's abstract of the patent:

The system includes a communications channel coupled to at least one terminal having an encoding device and to at least one terminal having a decoding device. A message-to-be-transferred is enciphered to ciphertext at the encoding terminal by encoding the message as a number M in a predetermined set. That number is then raised to a first predetermined power (associated with the intended receiver) and finally computed. The remainder or residue, C , is... computed when the exponentiated number is divided by the product of two predetermined prime numbers (associated with the intended receiver).

A detailed description of the algorithm was published in August 1977, in Scientific American's Mathematical Games column.^{[2][21]} This preceded the patent's filing date of December 1977. Consequently, the patent had no legal standing outside the United States. Had Cocks' work been publicly known, a patent in the United States would not have been legal either.

When the patent was issued, terms of patent were 17 years. The patent was about to expire on 21 September 2000, but RSA Security released the algorithm to the public domain on 6 September 2000.^[28]

Operation

The RSA algorithm involves four steps: key generation, key distribution, public-key operation (used for encryption or verifying a signature), and private key operation (used for decryption or signing a message).

A basic principle behind RSA is the observation that it is practical to find three very large positive integers e , d , and n , such that for all integers x ($0 \leq x < n$), both $(x^e)^d$ and x have the same remainder when divided by n (they are congruent modulo n):

$$(x^e)^d \equiv x \pmod{n}.$$

However, when given only e and n , it is infeasible to compute e^{th} roots modulo n ; that is, for uniform random y ($0 \leq y < n$), it is extremely difficult to find x such that $x^e \equiv y \pmod{n}$.

The integers n and e form the public key and d is the private key. The modular exponentiation to the power of e is used in encryption and in verifying signatures, and exponentiation to the power of d is used in decryption and in signing messages.

Key generation

The keys for the RSA algorithm are generated in the following way:

1. Choose two distinct large prime numbers p and q .
 - To make factoring infeasible, p and q must be chosen at random from a large space of possibilities, such as all prime numbers between 2^{1023} and 2^{1024} (corresponding to a 2,048-bit key). Many different algorithms for prime selection are used in practice.^[29]
 - p and q are kept secret.
2. Compute $n = pq$.
 - n is used as the modulus for both the public and private keys. Its length, usually expressed in bits, is the key length.
 - n is released as part of the public key.
3. Compute $\lambda(n)$, where λ is Carmichael's totient function. Since $n = pq$, $\lambda(n) = \text{lcm}(\lambda(p), \lambda(q))$, and since p and q are prime, $\lambda(p) = \varphi(p) = p - 1$, and likewise $\lambda(q) = q - 1$. Hence $\lambda(n) = \text{lcm}(p - 1, q - 1)$.
 - The lcm may be calculated through the Euclidean algorithm, since $\text{lcm}(a, b) = \frac{|ab|}{\text{gcd}(a, b)}$.
 - $\lambda(n)$ is kept secret.
4. Choose an integer e such that $1 < e < \lambda(n)$ and $\text{gcd}(e, \lambda(n)) = 1$; that is, e and $\lambda(n)$ are coprime.
 - e having a short bit-length and small Hamming weight results in more efficient encryption – the most commonly chosen value for e is $2^{16} + 1 = 65\,537$. The smallest (and fastest) possible value for e is 3, but such a small value for e may expose vulnerabilities in insecure padding schemes.^{[30][a]}
 - e is released as part of the public key.
5. Determine d as $d \equiv e^{-1} \pmod{\lambda(n)}$; that is, d is the modular multiplicative inverse of e .

- This means: solve for d the equation $de \equiv 1 \pmod{\lambda(n)}$; d can be computed efficiently by using the extended Euclidean algorithm, since, thanks to e and $\lambda(n)$ being coprime, said equation is a form of Bézout's identity, where d is one of the coefficients.
- d is kept secret as the *private key exponent*.

The *public key* consists of the modulus n and the public exponent e . The *private key* consists of the private exponent d , which must be kept secret. p , q , and $\lambda(n)$ must also be kept secret because they can be used to calculate d . In fact, they can all be discarded after d has been computed.^[31]

In the original RSA paper,^[3] the Euler totient function $\varphi(n) = (p - 1)(q - 1)$ is used instead of $\lambda(n)$ for calculating the private exponent d . Since $\varphi(n)$ is always divisible by $\lambda(n)$, the algorithm works as well. The possibility of using Euler totient function results also from Lagrange's theorem applied to the multiplicative group of integers modulo pq . Thus any d satisfying $d \cdot e \equiv 1 \pmod{\varphi(n)}$ also satisfies $d \cdot e \equiv 1 \pmod{\lambda(n)}$. However, computing d modulo $\varphi(n)$ will sometimes yield a result that is larger than necessary (i.e. $d > \lambda(n)$). Most of the implementations of RSA will accept exponents generated using either method (if they use the private exponent d at all, rather than using the optimized decryption method based on the Chinese remainder theorem described below), but some standards such as FIPS 186-4 (<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf#page=63>) (Section B.3.1) may require that $d < \lambda(n)$. Any "oversized" private exponents not meeting this criterion may always be reduced modulo $\lambda(n)$ to obtain a smaller equivalent exponent.

Note: The authors of the original RSA paper carry out the key generation by choosing d and then computing e as the modular multiplicative inverse of d modulo $\varphi(n)$, whereas most current implementations of RSA, such as those following PKCS#1, do the reverse—choose e and compute d from it. Since e can safely be small and fixed, whereas d must be chosen from a large enough space to resist attack, the modern approach can reduce the cost of the public-key operation without loss of security.^{[3][32]}

Key distribution

Suppose that Bob wants to send secret messages to Alice, or verify messages from Alice. If they decide to use RSA, Bob must know Alice's public key to encrypt his secret messages or verify Alice's messages, and Alice must use her private key to decrypt Bob's secret messages or sign her own messages.

To enable Bob to send his encrypted messages or verify her future messages, Alice transmits her public key (n, e) to Bob via a reliable, but not necessarily secret, route. Alice's private key (d) is never distributed.

Encryption

After Bob obtains Alice's public key, he can send a message M to Alice.

To do it, he first turns M into an integer m , the padded plaintext, such that $0 \leq m < n$, by using an agreed-upon reversible protocol known as a padding scheme. He then computes the ciphertext c , using Alice's public key e , by:

$$c \equiv m^e \pmod{n}.$$



This can be done reasonably quickly, even for very large numbers, using modular exponentiation. Bob then transmits c to Alice. Note that at least nine values of m will yield a ciphertext c equal to m ,^[b] but this is very unlikely to occur in practice.

Decryption

Alice can recover m from c by using her private key exponent d by computing

$$c^d \equiv (m^e)^d \equiv m \pmod{n}.$$

Given m , she can recover the original message M by reversing the padding scheme, or discard it as corrupted if the padding is invalid.

Alice **must** discard m if the padding is invalid: if she reveals any information about m when it has invalid padding, an adversary could exploit this to decrypt (or sign) messages without knowing the private key, by sending her random or maliciously crafted ciphertexts and observing how she responds.^[33]

Example

Here is an example of RSA encryption and decryption, ignoring the details of padding:^[c]

1. Choose two distinct prime numbers, such as
 $p = 61$ and $q = 53$.
2. Compute $n = pq$ giving
 $n = 61 \times 53 = 3233$.
3. Compute the Carmichael's totient function of the product as $\lambda(n) = \text{lcm}(p-1, q-1)$ giving
 $\lambda(3233) = \text{lcm}(60, 52) = 780$.
4. Choose any number $1 < e < 780$ that is coprime to 780. Choosing a prime number for e leaves us only to check that e is not a divisor of 780.
Let $e = 17$.
5. Compute d , the modular multiplicative inverse of $e \pmod{\lambda(n)}$, yielding
 $d = 413$,
as $1 = (17 \times 413) \pmod{780}$.

The **public key** is $(n = 3233, e = 17)$. For a padded plaintext message m , the encryption function is

$$\begin{aligned} c(m) &= m^e \pmod{n} \\ &= m^{17} \pmod{3233}. \end{aligned}$$

The **private key** is $(n = 3233, d = 413)$. For an encrypted ciphertext c , the decryption function is

$$\begin{aligned} m(c) &= c^d \pmod{n} \\ &= c^{413} \pmod{3233}. \end{aligned}$$

For instance, in order to encrypt $m = 65$, one calculates

$$c = 65^{17} \pmod{3233} = 2790.$$



To decrypt $c = 2790$, one calculates

$$m = 2790^{413} \bmod 3233 = 65.$$

Both of these calculations can be computed efficiently using the square-and-multiply algorithm for modular exponentiation. In real-life situations the primes selected would be much larger; in our example it would be trivial to factor $n = 3233$ (obtained from the freely available public key) back to the primes p and q . e , also from the public key, is then inverted to get d , thus acquiring the private key.

Practical implementations use the Chinese remainder theorem to speed up the calculation using modulus of factors ($\bmod pq$ using $\bmod p$ and $\bmod q$).

The values d_p , d_q and q_{inv} , which are part of the private key are computed as follows:

$$\begin{aligned}d_p &= d \bmod (p - 1) = 413 \bmod (61 - 1) = 53, \\d_q &= d \bmod (q - 1) = 413 \bmod (53 - 1) = 49, \\q_{inv} &= q^{-1} \bmod p = 53^{-1} \bmod 61 = 38 \\&\Rightarrow (q_{inv} \times q) \bmod p = 38 \times 53 \bmod 61 = 1.\end{aligned}$$

Here is how d_p , d_q and q_{inv} are used for efficient decryption (encryption is efficient by choice of a suitable d and e pair):

$$\begin{aligned}m_1 &= c^{d_p} \bmod p = 2790^{53} \bmod 61 = 4, \\m_2 &= c^{d_q} \bmod q = 2790^{49} \bmod 53 = 12, \\h &= (q_{inv} \times (m_1 - m_2)) \bmod p = (38 \times -8) \bmod 61 = 1, \\m &= m_2 + h \times q = 12 + 1 \times 53 = 65.\end{aligned}$$

Signing

Suppose Alice wishes to send a signed message m to Bob. She produces a hash value $h = \text{hash}(m)$ of the message m , raises it to the power of d (modulo n), and attaches $s = h^d \bmod n$ as a "signature" to the message.

Verifying

When Bob receives the message m and signature s , he uses the same hash algorithm in conjunction with Alice's public key to compute $h = \text{hash}(m)$. He raises the signature s to the power of e (modulo n), and compares the resulting hash value with the message's hash value:

$$s^e \stackrel{?}{\equiv} h \pmod{n}$$

If the two agree, he knows that the author of the message was in possession of Alice's private key and that the message has not been tampered with since being sent.

This equation is satisfied when $s = h^d \bmod n$ because of exponentiation rules:

$$s^e = (h^d)^e = h^{de} = h^{ed} = (h^e)^d \equiv h \pmod{n}.$$



The modular exponentiation for signing and verification is the same underlying mathematics as for decryption and encryption, but all the other details of padding scheme for secure public-key encryption and hashing for secure digital signature are different.^[32]

The use of a hash, first proposed in 1978 by Michael O. Rabin in the related Rabin signature algorithm,^{[34][35]} and the security of the hash, is essential for security of the signature:^{[36][37]} if Alice and Bob skipped the hash, and Bob checked for $s^e \equiv m \pmod{n}$ instead, then anyone could forge the signature $s = 1$ on the message $m = 1$, or take two signed messages (m_1, s_1) and (m_2, s_2) from Alice and then forge a third by multiplication, $(m_1 m_2, s_1 s_2)$, without knowledge of the private key.

Proofs of correctness

Proof using Fermat's little theorem

The proof of the correctness of RSA is based on Fermat's little theorem, stating that $a^{p-1} \equiv 1 \pmod{p}$ for any integer a and prime p , not dividing a .^[note 1]

We want to show that

$$(m^e)^d \equiv m \pmod{pq}$$

for every integer m when p and q are distinct prime numbers and e and d are positive integers satisfying $ed \equiv 1 \pmod{\lambda(pq)}$.

Since $\lambda(pq) = \text{lcm}(p-1, q-1)$ is, by construction, divisible by both $p-1$ and $q-1$, we can write

$$ed - 1 = h(p-1) = k(q-1)$$

for some nonnegative integers h and k .^[note 2]

To check whether two numbers, such as m^{ed} and m , are congruent mod pq , it suffices (and in fact is equivalent) to check that they are congruent mod p and mod q separately.^[note 3]

To show $m^{ed} \equiv m \pmod{p}$, we consider two cases:

1. If $m \equiv 0 \pmod{p}$, m is a multiple of p . Thus m^{ed} is a multiple of p . So $m^{ed} \equiv 0 \equiv m \pmod{p}$.
2. If $m \not\equiv 0 \pmod{p}$,

$$m^{ed} = m^{ed-1} m = m^{h(p-1)} m = (m^{p-1})^h m \equiv 1^h m \equiv m \pmod{p},$$

where we used Fermat's little theorem to replace $m^{p-1} \pmod{p}$ with 1.

The verification that $m^{ed} \equiv m \pmod{q}$ proceeds in a completely analogous way:

1. If $m \equiv 0 \pmod{q}$, m^{ed} is a multiple of q . So $m^{ed} \equiv 0 \equiv m \pmod{q}$.
2. If $m \not\equiv 0 \pmod{q}$,

$$m^{ed} = m^{ed-1} m = m^{k(q-1)} m = (m^{q-1})^k m \equiv 1^k m \equiv m \pmod{q}.$$

This completes the proof that, for any integer m , and integers e, d such that $ed \equiv 1 \pmod{\lambda}$

$$(m^e)^d \equiv m \pmod{pq}.$$



Notes

1. We cannot trivially break RSA by applying the theorem (mod pq) because pq is not prime.
2. In particular, the statement above holds for any e and d that satisfy $ed \equiv 1 \pmod{(p-1)(q-1)}$, since $(p-1)(q-1)$ is divisible by $\lambda(pq)$, and thus trivially also by $p-1$ and $q-1$. However, in modern implementations of RSA, it is common to use a reduced private exponent d that only satisfies the weaker, but sufficient condition $ed \equiv 1 \pmod{\lambda(pq)}$.
3. This is part of the Chinese remainder theorem, although it is not the significant part of that theorem.

Proof using Euler's theorem

Although the original paper of Rivest, Shamir, and Adleman used Fermat's little theorem to explain why RSA works, it is common to find proofs that rely instead on Euler's theorem.

We want to show that $m^{ed} \equiv m \pmod{n}$, where $n = pq$ is a product of two different prime numbers, and e and d are positive integers satisfying $ed \equiv 1 \pmod{\varphi(n)}$. Since e and d are positive, we can write $ed = 1 + h\varphi(n)$ for some non-negative integer h . Assuming that m is relatively prime to n , we have

$$m^{ed} = m^{1+h\varphi(n)} = m(m^{\varphi(n)})^h \equiv m(1)^h \equiv m \pmod{n},$$

where the second-last congruence follows from Euler's theorem.

More generally, for any e and d satisfying $ed \equiv 1 \pmod{\lambda(n)}$, the same conclusion follows from Carmichael's generalization of Euler's theorem, which states that $m^{\lambda(n)} \equiv 1 \pmod{n}$ for all m relatively prime to n .

When m is not relatively prime to n , the argument just given is invalid. This is highly improbable (only a proportion of $1/p + 1/q - 1/(pq)$ numbers have this property), but even in this case, the desired congruence is still true. Either $m \equiv 0 \pmod{p}$ or $m \equiv 0 \pmod{q}$, and these cases can be treated using the previous proof.

Padding

Attacks against plain RSA

There are a number of attacks against plain RSA as described below.

- When encrypting with low encryption exponents (e.g., $e = 3$) and small values of the m (i.e., $m < n^{1/e}$), the result of m^e is strictly less than the modulus n . In this case, ciphertexts can be decrypted easily by taking the e th root of the ciphertext over the integers.
- If the same clear-text message is sent to e or more recipients in an encrypted way, and the receivers share the same exponent e , but different p , q , and therefore n , then it is easy to decrypt the original clear-text message via the Chinese remainder theorem. Johan Håstad noticed that this attack is possible even if the clear texts are not equal, but the attacker knows a linear relation between them.^[38] This attack was later improved by Don Coppersmith (Coppersmith's attack).^[39]

- Because RSA encryption is a deterministic encryption algorithm (i.e., has no random component) an attacker can successfully launch a chosen plaintext attack against the cryptosystem, by encrypting likely plaintexts under the public key and test whether they are equal to the ciphertext. A cryptosystem is called semantically secure if an attacker cannot distinguish two encryptions from each other, even if the attacker knows (or has chosen) the corresponding plaintexts. RSA without padding is not semantically secure.^[40]
- RSA has the property that the product of two ciphertexts is equal to the encryption of the product of the respective plaintexts. That is, $m_1^e m_2^e \equiv (m_1 m_2)^e \pmod{n}$. Because of this multiplicative property, a chosen-ciphertext attack is possible. E.g., an attacker who wants to know the decryption of a ciphertext $c \equiv m^e \pmod{n}$ may ask the holder of the private key d to decrypt an unsuspecting-looking ciphertext $c' \equiv cr^e \pmod{n}$ for some value r chosen by the attacker. Because of the multiplicative property, c' is the encryption of $mr \pmod{n}$. Hence, if the attacker is successful with the attack, they will learn $mr \pmod{n}$, from which they can derive the message m by multiplying mr with the modular inverse of r modulo n .^{[33][41]}
- Given the private exponent d , one can efficiently factor the modulus $n = pq$. And given factorization of the modulus $n = pq$, one can obtain any private key (d', n) generated against a public key (e', n) .^[30]

Padding schemes

To avoid these problems, practical RSA implementations typically embed some form of structured, randomized padding into the value m before encrypting it. This padding ensures that m does not fall into the range of insecure plaintexts, and that a given message, once padded, will encrypt to one of a large number of different possible ciphertexts.

Standards such as PKCS#1 have been carefully designed to securely pad messages prior to RSA encryption. Because these schemes pad the plaintext m with some number of additional bits, the size of the un-padded message M must be somewhat smaller. RSA padding schemes must be carefully designed so as to prevent sophisticated attacks that may be facilitated by a predictable message structure. Early versions of the PKCS#1 standard (up to version 1.5) used a construction that appears to make RSA semantically secure. However, at Crypto 1998, Bleichenbacher showed that this version is vulnerable to a practical adaptive chosen-ciphertext attack. Furthermore, at Eurocrypt 2000, Coron et al.^[42] showed that for some types of messages, this padding does not provide a high enough level of security. Later versions of the standard include Optimal Asymmetric Encryption Padding (OAEP), which prevents these attacks. As such, OAEP should be used in any new application, and PKCS#1 v1.5 padding should be replaced wherever possible. The PKCS#1 standard also incorporates processing schemes designed to provide additional security for RSA signatures, e.g. the Probabilistic Signature Scheme for RSA (RSA-PSS).

Secure padding schemes such as RSA-PSS are as essential for the security of message signing as they are for message encryption. Two USA patents on PSS were granted (U.S. patent 6,266,771 (<https://patents.google.com/patent/US6266771>) and U.S. patent 7,036,014 (<https://patents.google.com/patent/US7036014>)); however, these patents expired on 24 July 2009 and 25 April 2010 respectively. Use of PSS no longer seems to be encumbered by patents. Note that using different RSA key pairs for encryption and signing is potentially more secure.^[43]



Security and practical considerations

Using the Chinese remainder algorithm

For efficiency, many popular crypto libraries (such as [OpenSSL](#), [Java](#) and [.NET](#)) use for decryption and signing the following optimization based on the [Chinese remainder theorem](#).^[44] The following values are precomputed and stored as part of the private key:

- p and q – the primes from the key generation,
- $d_P = d \pmod{p-1}$,
- $d_Q = d \pmod{q-1}$,
- $q_{inv} = q^{-1} \pmod{p}$.

These values allow the recipient to compute the exponentiation $m = c^d \pmod{pq}$ more efficiently as follows:

$$\begin{aligned} m_1 &= c^{d_P} \pmod{p}, \\ m_2 &= c^{d_Q} \pmod{q}, \\ h &= q_{inv}(m_1 - m_2) \pmod{p},^{[d]} \\ m &= m_2 + hq. \end{aligned}$$

This is more efficient than computing [exponentiation by squaring](#), even though two modular exponentiations have to be computed. The reason is that these two modular exponentiations both use a smaller exponent and a smaller modulus.

Integer factorization and the RSA problem

The security of the RSA cryptosystem is based on two mathematical problems: the problem of [factoring large numbers](#) and the [RSA problem](#). Full decryption of an RSA ciphertext is thought to be infeasible on the assumption that both of these problems are [hard](#), i.e., no efficient algorithm exists for solving them. Providing security against *partial* decryption may require the addition of a [secure padding scheme](#).^[45]

The [RSA problem](#) is defined as the task of taking e th roots modulo a composite n : recovering a value m such that $c \equiv m^e \pmod{n}$, where (n, e) is an RSA public key, and c is an RSA ciphertext. Currently the most promising approach to solving the RSA problem is to factor the modulus n . With the ability to recover prime factors, an attacker can compute the secret exponent d from a public key (n, e) , then decrypt c using the standard procedure. To accomplish this, an attacker factors n into p and q , and computes $\text{lcm}(p-1, q-1)$ that allows the determination of d from e . No polynomial-time method for factoring large integers on a classical computer has yet been found, but it has not been proven that none exists; see [integer factorization](#) for a discussion of this problem.

The first RSA-512 factorization in 1999 used hundreds of computers and required the equivalent of 8,400 MIPS years, over an elapsed time of about seven months.^[46] By 2009, Benjamin Moody could factor a 512-bit RSA key in 73 days using only public software (GGNFS) and his desktop computer (a dual-core [Athlon64](#) with a 1,900 MHz CPU). Just less than 5 gigabytes of disk space was required and about 2.5 gigabytes of RAM for the sieving process.

Rivest, Shamir, and Adleman noted^[3] that Miller has shown that – assuming the truth of the extended Riemann hypothesis – finding d from n and e is as hard as factoring n into p and q (up to a polynomial time difference).^[47] However, Rivest, Shamir, and Adleman noted, in section IX/D of their paper, that they had not found a proof that inverting RSA is as hard as factoring.

As of 2020, the largest publicly known factored RSA number had 829 bits (250 decimal digits, RSA-250).^[48] Its factorization, by a state-of-the-art distributed implementation, took about 2,700 CPU-years. In practice, RSA keys are typically 1024 to 4096 bits long. In 2003, RSA Security estimated that 1024-bit keys were likely to become crackable by 2010.^[49] As of 2020, it is not known whether such keys can be cracked, but minimum recommendations have moved to at least 2048 bits.^[50] It is generally presumed that RSA is secure if n is sufficiently large, outside of quantum computing.

If n is 300 bits or shorter, it can be factored in a few hours on a personal computer, using software already freely available. Keys of 512 bits have been shown to be practically breakable in 1999, when RSA-155 was factored by using several hundred computers, and these are now factored in a few weeks using common hardware. Exploits using 512-bit code-signing certificates that may have been factored were reported in 2011.^[51] A theoretical hardware device named TWIRL, described by Shamir and Tromer in 2003, called into question the security of 1024-bit keys.^[49]

In 1994, Peter Shor showed that a quantum computer – if one could ever be practically created for the purpose – would be able to factor in polynomial time, breaking RSA; see Shor's algorithm.

Faulty key generation

Finding the large primes p and q is usually done by testing random numbers of the correct size with probabilistic primality tests that quickly eliminate virtually all of the nonprimes.

The numbers p and q should not be "too close", lest the Fermat factorization for n be successful. If $p - q$ is less than $2n^{1/4}$ ($n = p \cdot q$, which even for "small" 1024-bit values of n is 3×10^{77}), solving for p and q is trivial. Furthermore, if either $p - 1$ or $q - 1$ has only small prime factors, n can be factored quickly by Pollard's $p - 1$ algorithm, and hence such values of p or q should be discarded.

It is important that the private exponent d be large enough. Michael J. Wiener showed that if p is between q and $2q$ (which is quite typical) and $d < n^{1/4}/3$, then d can be computed efficiently from n and e .^[52]

There is no known attack against small public exponents such as $e = 3$, provided that the proper padding is used. Coppersmith's attack has many applications in attacking RSA specifically if the public exponent e is small and if the encrypted message is short and not padded. 65537 is a commonly used value for e ; this value can be regarded as a compromise between avoiding potential small-exponent attacks and still allowing efficient encryptions (or signature verification). The NIST Special Publication on Computer Security (SP 800-78 Rev. 1 of August 2007) does not allow public exponents e smaller than 65537, but does not state a reason for this restriction.

In October 2017, a team of researchers from Masaryk University announced the ROCA vulnerability, which affects RSA keys generated by an algorithm embodied in a library from Infineon known as RSALib. A large number of smart cards and trusted platform modules (TPM) were shown to be affected. Vulnerable RSA keys are easily identified using a test program that was released.^[53]

Importance of strong random number generation

A cryptographically strong random number generator, which has been properly seeded with adequate entropy, must be used to generate the primes p and q . An analysis comparing millions of public keys gathered from the Internet was carried out in early 2012 by Arjen K. Lenstra, James P. Hughes, Maxime Augier, Joppe W. Bos, Thorsten Kleinjung and Christophe Wachter. They were able to factor 0.2% of the keys using only Euclid's algorithm.^{[54][55]}

They exploited a weakness unique to cryptosystems based on integer factorization. If $n = pq$ is one public key, and $n' = p'q'$ is another, then if by chance $p = p'$ (but q is not equal to q'), then a simple computation of $\gcd(n, n') = p$ factors both n and n' , totally compromising both keys. Lenstra et al. note that this problem can be minimized by using a strong random seed of bit length twice the intended security level, or by employing a deterministic function to choose q given p , instead of choosing p and q independently.

Nadia Heninger was part of a group that did a similar experiment. They used an idea of Daniel J. Bernstein to compute the GCD of each RSA key n against the product of all the other keys n' they had found (a 729-million-digit number), instead of computing each $\gcd(n, n')$ separately, thereby achieving a very significant speedup, since after one large division, the GCD problem is of normal size.

Heninger says in her blog that the bad keys occurred almost entirely in embedded applications, including "firewalls, routers, VPN devices, remote server administration devices, printers, projectors, and VOIP phones" from more than 30 manufacturers. Heninger explains that the one-shared-prime problem uncovered by the two groups results from situations where the pseudorandom number generator is poorly seeded initially, and then is reseeded between the generation of the first and second primes. Using seeds of sufficiently high entropy obtained from key stroke timings or electronic diode noise or atmospheric noise from a radio receiver tuned between stations should solve the problem.^[56]

Strong random number generation is important throughout every phase of public-key cryptography. For instance, if a weak generator is used for the symmetric keys that are being distributed by RSA, then an eavesdropper could bypass RSA and guess the symmetric keys directly.

Timing attacks

Kocher described a new attack on RSA in 1995: if the attacker Eve knows Alice's hardware in sufficient detail and is able to measure the decryption times for several known ciphertexts, Eve can deduce the decryption key d quickly. This attack can also be applied against the RSA signature scheme. In 2003, Boneh and Brumley demonstrated a more practical attack capable of recovering RSA factorizations over a network connection (e.g., from a Secure Sockets Layer (SSL)-enabled webserver).^[57] This attack takes advantage of information leaked by the Chinese remainder theorem optimization used by many RSA implementations.

One way to thwart these attacks is to ensure that the decryption operation takes a constant amount of time for every ciphertext. However, this approach can significantly reduce performance. Instead, most RSA implementations use an alternate technique known as cryptographic blinding. RSA blinding makes use of the multiplicative property of RSA. Instead of computing $c^d \pmod{n}$, Alice first chooses a secret random value r and computes $(r^e c)^d \pmod{n}$. The result of this computation, after applying Euler's theorem, is $rc^d \pmod{n}$, and so the effect of r can be removed by

multiplying by its inverse. A new value of r is chosen for each ciphertext. With blinding applied, the decryption time is no longer correlated to the value of the input ciphertext, and so the timing attack fails.

Adaptive chosen-ciphertext attacks

In 1998, Daniel Bleichenbacher described the first practical adaptive chosen-ciphertext attack against RSA-encrypted messages using the PKCS #1 v1 padding scheme (a padding scheme randomizes and adds structure to an RSA-encrypted message, so it is possible to determine whether a decrypted message is valid). Due to flaws with the PKCS #1 scheme, Bleichenbacher was able to mount a practical attack against RSA implementations of the Secure Sockets Layer protocol and to recover session keys. As a result of this work, cryptographers now recommend the use of provably secure padding schemes such as Optimal Asymmetric Encryption Padding, and RSA Laboratories has released new versions of PKCS #1 that are not vulnerable to these attacks.

A variant of this attack, dubbed "BERserk", came back in 2014.^{[58][59]} It impacted the Mozilla NSS Crypto Library, which was used notably by Firefox and Chrome.

Side-channel analysis attacks

A side-channel attack using branch-prediction analysis (BPA) has been described. Many processors use a branch predictor to determine whether a conditional branch in the instruction flow of a program is likely to be taken or not. Often these processors also implement simultaneous multithreading (SMT). Branch-prediction analysis attacks use a spy process to discover (statistically) the private key when processed with these processors.

Simple Branch Prediction Analysis (SBPA) claims to improve BPA in a non-statistical way. In their paper, "On the Power of Simple Branch Prediction Analysis",^[60] the authors of SBPA (Onur Aciicmez and Cetin Kaya Koc) claim to have discovered 508 out of 512 bits of an RSA key in 10 iterations.

Fault injection attack

A power-fault attack on RSA implementations was described in 2010.^[61] The author recovered the key by varying the CPU power voltage outside limits; this caused multiple power faults on the server.

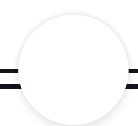
The CRT implementation is sensitive to fault injection attacks. If an attacker can obtain 1 faulty signature, the private key can be calculated.^[62]

Tricky implementation

There are many details to keep in mind in order to implement RSA securely (strong PRNG, acceptable public exponent, etc.). This makes the implementation challenging, to the point that the book *Practical Cryptography With Go* suggests avoiding RSA if possible.^[63]

Implementations

Some cryptography libraries that provide support for RSA include:



- [Botan](#)
- [Bouncy Castle](#)
- [cryptlib](#)
- [Crypto++](#)
- [Libgcrypt](#)
- [Nettle](#)
- [OpenSSL](#)
- [wolfCrypt](#)
- [GnuTLS](#)
- [mbed TLS](#)
- [LibreSSL](#)

See also



- [Acoustic cryptanalysis](#)
- [Computational complexity theory](#)
- [Diffie–Hellman key exchange](#)
- [Digital Signature Algorithm](#)
- [Elliptic-curve cryptography](#)
- [Key exchange](#)
- [Key management](#)
- [Key size](#)
- [Public-key cryptography](#)
- [Rabin signature](#)
- [Trapdoor function](#)

Notes

- $e = 2$ is also possible (and even faster) but qualitatively different because squaring is not a permutation; this is the basis of the [Rabin signature algorithm](#).
- Namely, the values of m which are equal to -1 , 0 , or 1 modulo p while also equal to -1 , 0 , or 1 modulo q . There will be more values of m having $c = m$ if $p - 1$ or $q - 1$ has other divisors in common with $e - 1$ besides 2 because this gives more values of m such that $m^{e-1} \bmod p = 1$ or $m^{e-1} \bmod q = 1$ respectively.
- The parameters used here are artificially small, but one can also [OpenSSL](#) can also be used to generate and examine a real keypair.
- If $m_1 < m_2$, then some libraries compute h as $q_{\text{inv}} \left[\left(m_1 + \left\lceil \frac{q}{p} \right\rceil p \right) - m_2 \right] \pmod{p}$.

References

- Rivest, R.L.; Shamir, A.; Adleman, L. (1977). *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems* (<https://publications.csail.mit.edu/lcs/pubs/pdf/MIT-LCS-TM-082.pdf>) (PDF) (Technical report). MIT Laboratory for Computer Science. hdl:1721.1/148910 (<https://hdl.handle.net/1721.1/148910>). MIT-LCS-TM-082.

2. Gardner, Martin (August 1977). "Mathematical Games: A new kind of cipher that would take millions of years to break" (https://web.archive.org/web/20250711155721/https://simson.net/ref/1977/Gardner_RSA.pdf) (PDF). *Scientific American*. Vol. 237, no. 2. doi:10.1038/scientificamerican0877-120 (<https://doi.org/10.1038%2Fscientificamerican0877-120>). Archived from the original (https://simson.net/ref/1977/Gardner_RSA.pdf) (PDF) on 2025-07-11.
3. Rivest, R.; Shamir, A.; Adleman, L. (February 1978). "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems" (<https://web.archive.org/web/20230127011251/http://people.csail.mit.edu/rivest/Rsapaper.pdf>) (PDF). *Communications of the ACM*. **21** (2): 120–126. CiteSeerX 10.1.1.607.2677 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.607.2677>). doi:10.1145/359340.359342 (<https://doi.org/10.1145%2F359340.359342>). Archived from the original (<http://people.csail.mit.edu/rivest/Rsapaper.pdf>) (PDF) on 2023-01-27. Retrieved 2025-07-30.
4. Smart, Nigel (February 19, 2008). "Dr Clifford Cocks CB" (<https://www.bristol.ac.uk/alumni/our-alumni/honorary-degrees/honorary-graduates/2008/cocks.html>). Bristol University. Retrieved June 20, 2025.
5. Bellare, Mihir; Rogaway, Phillip. Maurer, Ueli (ed.). *The exact security of digital signatures: How to sign with RSA and Rabin* (<https://www.cs.ucdavis.edu/~rogaway/papers/exact.html>). Advances in Cryptology—EUROCRYPT '96 (<https://link.springer.com/book/10.1007/3-540-68339-9>). Lecture Notes in Computer Science. Springer. pp. 399–416. doi:10.1007/3-540-68339-9_34 (https://doi.org/10.1007%2F3-540-68339-9_34). ISBN 978-3-540-61186-8.
6. Aumasson, Jean-Philippe (2018). "10. RSA: Signing with RSA". *Serious Cryptography*. No Starch Press. pp. 188–191. ISBN 978-1-59327-826-7.
7. Stinson, Douglas (2006). "7: Signature Schemes". *Cryptography: Theory and Practice* (3rd ed.). Chapman & Hall/CRC. pp. 281–318. ISBN 978-1-58488-508-5.
8. Ferguson, Niels; Kohno, Tadayoshi; Schneier, Bruce (2010). "12. RSA". *Cryptography Engineering*. Wiley. pp. 195–211. ISBN 978-0-470-47424-2.
9. Galbraith, Steven (2012). "§ 24.6: Digital signatures based on RSA and Rabin". *Mathematics of Public-Key Cryptography*. Cambridge University Press. pp. 7–9. ISBN 978-1-107-01392-6.
10. B. Kaliski; A. Rusch; J. Johnsson; A. Rusch (November 2016). K. Moriarty (ed.). *PKCS #1: RSA Cryptography Specifications Version 2.2* (<https://www.rfc-editor.org/rfc/rfc8017>). Internet Engineering Task Force. doi:10.17487/RFC8017 (<https://doi.org/10.17487%2FRFC8017>). ISSN 2070-1721 (<https://search.worldcat.org/issn/2070-1721>). RFC 8017 (<https://datatracker.ietf.org/doc/html/rfc8017>). *Informational*. Obsoletes RFC 3447 (<https://www.rfc-editor.org/rfc/rfc3447>).
11. Bellare, Mihir; Rogaway, Phillip. Santis, Alfredo (ed.). *Optimal asymmetric encryption*. Advances in Cryptology—EUROCRYPT '94 (<https://link.springer.com/book/10.1007/BFb0053418>). Lecture Notes in Computer Science. Springer. pp. 92–111. doi:10.1007/BFb0053428 (<https://doi.org/10.1007%2FBFb0053428>). ISBN 978-3-540-60176-0.
12. Aumasson, Jean-Philippe (2018). "10. RSA: Encrypting with RSA". *Serious Cryptography*. No Starch Press. pp. 185–188. ISBN 978-1-59327-826-7.
13. Galbraith, Steven (2012). "§24.7: Public-key encryption based on RSA and Rabin". *Mathematics of Public-Key Cryptography*. Cambridge University Press. pp. 511–512. ISBN 978-1-107-01392-6.
14. Shoup, Victor (2001), *A Proposal for an ISO Standard for Public Key Encryption (version 2.1)* (<https://eprint.iacr.org/2001/112>), Cryptology ePrint Archive, International Association for Cryptologic Research
15. Ferguson, Niels; Kohno, Tadayoshi; Schneier, Bruce (2010). "12. RSA". *Cryptography Engineering*. Wiley. pp. 195–211. ISBN 978-0-470-47424-2.
16. R. Housley; S. Turner (February 2025). *Use of the RSA-KEM Algorithm in the Cryptographic Message Syntax (CMS)* (<https://www.rfc-editor.org/rfc/rfc9690>). Internet Engineering Task Force. doi:10.17487/RFC9690 (<https://doi.org/10.17487%2FRFC9690>). RFC 9690 (<https://datatracker.ietf.org/doc/html/rfc9690>). *Proposed Standard*. Obsoletes RFC 5990 (<https://www.rfc-editor.org/rfc/rfc5990>).

17. Castelvechi, Davide (2020-10-30). "Quantum-computing pioneer warns of complacency over Internet security" (<https://www.nature.com/articles/d41586-020-03068-9>). *Nature*. **587** (7833): 189. Bibcode:2020Natur.587..189C (<https://ui.adsabs.harvard.edu/abs/2020Natur.587..189C>). doi:10.1038/d41586-020-03068-9 (<https://doi.org/10.1038%2Fd41586-020-03068-9>). PMID 33139910 (<https://pubmed.ncbi.nlm.nih.gov/33139910>). S2CID 226243008 (<https://api.semanticscholar.org/CorpusID:226243008>). 2020 interview of Peter Shor.
18. Diffie, W.; Hellman, M. E. (November 1976). "New directions in cryptography" (<https://web.archive.org/web/20141129035850/https://ee.stanford.edu/%7Ehellman/publications/24.pdf>) (PDF). *IEEE Transactions on Information Theory*. **22** (6): 644–654. Bibcode:1976ITIT...22..644D (<https://ui.adsabs.harvard.edu/abs/1976ITIT...22..644D>). CiteSeerX 10.1.1.37.9720 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.37.9720>). doi:10.1109/TIT.1976.1055638 (<https://doi.org/10.1109%2FTIT.1976.1055638>). ISSN 0018-9448 (<https://search.worldcat.org/issn/0018-9448>). Archived from the original (<http://ee.stanford.edu/%7Ehellman/publications/24.pdf>) (PDF) on 2014-11-29. Retrieved 2025-07-30.
19. Rivest, Ronald. "The Early Days of RSA – History and Lessons" (<https://people.csail.mit.edu/rivest/pubs/ARS03.rivest-slides.pdf>) (PDF).
20. Calderbank, Michael (2007-08-20). "The RSA Cryptosystem: History, Algorithm, Primes" (<http://www.math.uchicago.edu/~may/VIGRE/VIGRE2007/REUPapers/FINALAPP/Calderbank.pdf>) (PDF).
21. Robinson, Sara (June 2003). "Still Guarding Secrets after Years of Attacks, RSA Earns Accolades for its Founders" (<https://web.archive.org/web/20221215130836/https://www.msri.org/people/members/sara/articles/rsa.pdf>) (PDF). *SIAM News*. **36** (5). Archived from the original (<http://www.msri.org/people/members/sara/articles/rsa.pdf>) (PDF) on 2022-12-15.
22. Cocks, C. C. (20 November 1973). "A Note on Non-Secret Encryption" (https://web.archive.org/web/20180928121748/https://www.gchq.gov.uk/sites/default/files/document_files/Cliff%20Cocks%20paper%2019731120.pdf) (PDF). *www.gchq.gov.uk*. Archived from the original (https://www.gchq.gov.uk/sites/default/files/document_files/Cliff%20Cocks%20paper%2019731120.pdf) (PDF) on 28 September 2018. Retrieved 2017-05-30.
23. Jim Sauerberg. "From Private to Public Key Ciphers in Three Easy Steps" (https://ww2.amstat.org/mam/06/Sauerberg_PKC-essay.html).
24. Margaret Cozzens and Steven J. Miller. "The Mathematics of Encryption: An Elementary Introduction" (<https://books.google.com/books?id=GbKyAAAAQBAJ>). p. 180.
25. Alasdair McAndrew. "Introduction to Cryptography with Open-Source Software" (<https://books.google.com/books?id=9ITRBQAAQBAJ>). p. 12.
26. Surender R. Chiluka. "Public key Cryptography" (<https://web.archive.org/web/20220319203917/https://www.cs.uri.edu/cryptography/publickeykidkrypto.htm>).
27. Neal Koblitz. "Cryptography As a Teaching Tool" (<https://sites.math.washington.edu/~koblitz/crlogia.html>). *Cryptologia*, Vol. 21, No. 4 (1997).
28. "RSA Security Releases RSA Encryption Algorithm into Public Domain" (https://web.archive.org/web/20070621021111/http://www.rsa.com/press_release.aspx?id=261). Archived from the original (http://www.rsa.com/press_release.aspx?id=261) on June 21, 2007. Retrieved 2010-03-03.
29. Švenda, Petr; Nemec, Matúš; Sekan, Peter; Kvašňovský, Rudolf; Formánek, David; Komárek, David; Matyáš, Vashek (August 2016). *The Million-Key Question—Investigating the Origins of RSA Public Keys* (<https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/svenda>). 25th USENIX Security Symposium. Austin, TX, United States: USENIX Association. pp. 893–910. ISBN 978-1-931971-32-4.
30. Boneh, Dan (1999). "Twenty Years of attacks on the RSA Cryptosystem" (<http://crypto.stanford.edu/~dabo/abstracts/RSAattack-survey.html>). *Notices of the American Mathematical Society*. **46** (2): 203–213.
31. Applied Cryptography, John Wiley & Sons, New York, 1996. Bruce Schneier, p. 467.

32. Johnson, J.; Kaliski, B. (February 2003). *Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1* (<https://www.rfc-editor.org/rfc/rfc3447>). Network Working Group. doi:10.17487/RFC3447 (<https://doi.org/10.17487%2FRFC3447>). RFC 3447 (<https://datatracker.ietf.org/doc/html/rfc3447>). Retrieved 9 March 2016.
33. Bleichenbacher, Daniel (1998). Krawczyk, Hugo (ed.). *Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1*. Advances in Cryptology—CRYPTO '98 (<https://link.springer.com/book/10.1007/BFb0055715>). Lecture Notes in Computer Science. Springer. pp. 1–12. doi:10.1007/BFb0055716 (<https://doi.org/10.1007%2BFb0055716>). ISBN 978-3-540-68462-6.
34. Rabin, Michael O. (1978). "Digitalized Signatures". In DeMillo, Richard A.; Dobkin, David P.; Jones, Anita K.; Lipton, Richard J. (eds.). *Foundations of Secure Computation*. New York: Academic Press. pp. 155–168. ISBN 0-12-210350-5.
35. Rabin, Michael O. (January 1979). *Digitalized Signatures and Public Key Functions as Intractable as Factorization* (<http://publications.csail.mit.edu/lcs/pubs/pdf/MIT-LCS-TR-212.pdf>) (PDF) (Technical report). Cambridge, MA, United States: MIT Laboratory for Computer Science. TR-212.
36. Bernstein, Daniel J. (January 31, 2008). RSA signatures and Rabin–Williams signatures: the state of the art (<https://cr.yp.to/papers.html#rwsota>) (Report). (additional information at <https://cr.yp.to/sigs.html>)
37. Bellare, Mihir; Rogaway, Phillip (May 1996). Maurer, Ueli (ed.). *The Exact Security of Digital Signatures—How to Sign with RSA and Rabin*. Advances in Cryptology – EUROCRYPT '96 (<https://link.springer.com/book/10.1007/3-540-68339-9>). Lecture Notes in Computer Science. Vol. 1070. Saragossa, Spain: Springer. pp. 399–416. doi:10.1007/3-540-68339-9_34 (https://doi.org/10.1007%2F3-540-68339-9_34). ISBN 978-3-540-61186-8.
38. Håstad, Johan (1986). "On using RSA with Low Exponent in a Public Key Network". *Advances in Cryptology – CRYPTO '85 Proceedings*. Lecture Notes in Computer Science. Vol. 218. pp. 403–408. doi:10.1007/3-540-39799-X_29 (https://doi.org/10.1007%2F3-540-39799-X_29). ISBN 978-3-540-16463-0.
39. Coppersmith, Don (1997). "Small Solutions to Polynomial Equations, and Low Exponent RSA Vulnerabilities" (<https://www.di.ens.fr/~fouque/ens-rennes/coppersmith.pdf>) (PDF). *Journal of Cryptology*. **10** (4): 233–260. CiteSeerX 10.1.1.298.4806 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.298.4806>). doi:10.1007/s001459900030 (<https://doi.org/10.1007%2Fs001459900030>). S2CID 15726802 (<https://api.semanticscholar.org/CorpusID:15726802>).
40. Goldwasser, Shafi; Micali, Silvio (1982-05-05). "Probabilistic encryption & how to play mental poker keeping secret all partial information" (<https://doi.org/10.1145/800070.802212>). *Proceedings of the fourteenth annual ACM symposium on Theory of computing - STOC '82*. New York, NY, USA: Association for Computing Machinery. pp. 365–377. doi:10.1145/800070.802212 (<https://doi.org/10.1145%2F800070.802212>). ISBN 978-0-89791-070-5. S2CID 10316867 (<https://api.semanticscholar.org/CorpusID:10316867>).
41. Davida, George I. (1982). *Chosen signature cryptanalysis of the RSA (MIT) public key cryptosystem* (Technical report). Department of Electrical Engineering and Computer Science, University of Wisconsin, Milwaukee. Technical Report TR-CS-82-2.
42. Coron, Jean-Sébastien; Joye, Marc; Naccache, David; Paillier, Pascal (2000). "New Attacks on PKCS#1 v1.5 Encryption". In Preneel, Bart (ed.). *Advances in Cryptology — EUROCRYPT 2000*. Lecture Notes in Computer Science. Vol. 1807. Berlin, Heidelberg: Springer. pp. 369–381. doi:10.1007/3-540-45539-6_25 (https://doi.org/10.1007%2F3-540-45539-6_25). ISBN 978-3-540-45539-4.
43. "RSA Algorithm" (https://www.di-mgt.com.au/rsa_alg.html#weaknesses).
44. "OpenSSL bn_s390x.c" (https://github.com/openssl/openssl/blob/422a13fb5cd668cdc4c1eebc8accb4d25c3d8eb/crypto/bn/bn_s390x.c#L70). *Github*. Retrieved 2 August 2024.
45. Machie, Edmond K. (29 March 2013). *Network security traceback attack and react in the United States Department of Defense network* (<https://books.google.com/books?id=AK5M...Zb buMC&pg=PA167>). Trafford. p. 167. ISBN 978-1466985742.

46. Lenstra, Arjen; et al. (Group) (2000). "Factorization of a 512-bit RSA Modulus" (<https://www.iacr.org/archive/eurocrypt2000/1807/18070001-new.pdf>) (PDF). Eurocrypt.
47. Miller, Gary L. (1975). "Riemann's Hypothesis and Tests for Primality" (<https://www.cs.cmu.edu/~glmiller/Publications/Papers/Mi75.pdf>) (PDF). *Proceedings of Seventh Annual ACM Symposium on Theory of Computing*. pp. 234–239.
48. Zimmermann, Paul (2020-02-28). "Factorization of RSA-250" (<https://web.archive.org/web/20200228234716/https://lists.gforge.inria.fr/pipermail/cado-nfs-discuss/2020-February/001166.html>). Cado-nfs-discuss. Archived from the original (<https://lists.gforge.inria.fr/pipermail/cado-nfs-discuss/2020-February/001166.html>) on 2020-02-28. Retrieved 2020-07-12.
49. Kaliski, Burt (2003-05-06). "TWIRL and RSA Key Size" (<https://web.archive.org/web/20170417095741/https://www.emc.com/emc-plus/rsa-labs/historical/twirl-and-rsa-key-size.htm>). RSA Laboratories. Archived from the original (<http://emc.com/emc-plus/rsa-labs/historical/twirl-and-rsa-key-size.htm>) on 2017-04-17. Retrieved 2017-11-24.
50. Barker, Elaine; Dang, Quynh (2015-01-22). "NIST Special Publication 800-57 Part 3 Revision 1: Recommendation for Key Management: Application-Specific Key Management Guidance" (<http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf>) (PDF). National Institute of Standards and Technology. p. 12. doi:10.6028/NIST.SP.800-57pt3r1 (<https://doi.org/10.6028%2FNIST.SP.800-57pt3r1>). Retrieved 2017-11-24.
51. Sandee, Michael (November 21, 2011). "RSA-512 certificates abused in-the-wild" (<https://blog.fox-it.com/2011/11/21/rsa-512-certificates-abused-in-the-wild/>). *Fox-IT International blog*.
52. Wiener, Michael J. (May 1990). "Cryptanalysis of short RSA secret exponents" (<http://www.cits.rub.de/imperia/md/content/may/krypto2ss08/shortsecretexponents.pdf>) (PDF). *IEEE Transactions on Information Theory*. **36** (3): 553–558. Bibcode:1990ITIT...36..553W (<https://ui.adsabs.harvard.edu/abs/1990ITIT...36..553W>). doi:10.1109/18.54902 (<https://doi.org/10.1109%2F18.54902>). S2CID 7120331 (<https://api.semanticscholar.org/CorpusID:7120331>).
53. Nemec, Matus; Sys, Marek; Svenda, Petr; Klinec, Dusan; Matyas, Vashek (November 2017). "The Return of Coppersmith's Attack: Practical Factorization of Widely Used RSA Moduli" (http://crocs.fi.muni.cz/media/public/papers/nemec_roca_ccs17_preprint.pdf) (PDF). *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS '17. doi:10.1145/3133956.3133969 (<https://doi.org/10.1145%2F3133956.3133969>).
54. Markoff, John (February 14, 2012). "Flaw Found in an Online Encryption Method" (<https://www.nytimes.com/2012/02/15/technology/researchers-find-flaw-in-an-online-encryption-method.html>). *The New York Times*.
55. Lenstra, Arjen K.; Hughes, James P.; Augier, Maxime; Bos, Joppe W.; Kleinjung, Thorsten; Wachter, Christophe (2012). "Ron was wrong, Whit is right" (<http://eprint.iacr.org/2012/064.pdf>) (PDF).
56. Heninger, Nadia (February 15, 2012). "New research: There's no need to panic over factorable keys—just mind your Ps and Qs" (<https://freedom-to-tinker.com/blog/nadah/new-research-there-s-no-need-panic-over-factorable-keys-just-mind-your-ps-and-qs>). *Freedom to Tinker*.
57. Brumley, David; Boneh, Dan (2003). "Remote timing attacks are practical" (<http://crypto.stanford.edu/~dabo/papers/ssl-timing.pdf>) (PDF). *Proceedings of the 12th Conference on USENIX Security Symposium*. SSYM'03.
58. "'BERserk' Bug Uncovered In Mozilla NSS Crypto Library Impacts Firefox, Chrome" (<https://www.darkreading.com/attacks-breaches/-berserk-bug-uncovered-in-mozilla-nss-crypto-library-impacts-firefox-chrome>). *Dark Reading*. 25 September 2014. Retrieved 4 January 2022.
59. "RSA Signature Forgery in NSS" (<https://www.mozilla.org/en-US/security/advisories/mfsa2014-73/>). *Mozilla*.
60. Aciçmez, Onur; Koç, Çetin Kaya; Seifert, Jean-Pierre (2007). "On the power of simple branch prediction analysis". *Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security*. ASIACCS '07. pp. 312–320. CiteSeerX 10.1.1.80.1438 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.80.1438>). doi:10.1145/1229285.1266999 (<https://doi.org/10.1145%2F1229285.1266999>).

61. Pellegrini, Andrea; Bertacco, Valeria; Austin, Todd (March 2010). "Fault-based attack of RSA authentication". *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*. pp. 855–860. doi:[10.1109/DATE.2010.5456933](https://doi.org/10.1109/DATE.2010.5456933) (<https://doi.org/10.1109%2FDATE.2010.5456933>). ISBN 978-3-9810801-6-2.
62. Boneh, Dan; DeMillo, Richard A.; Lipton, Richard J. (Nov 2000). "On the Importance of Eliminating Errors in Cryptographic Computations". *Journal of Cryptology*. **14** (2): 106–107. doi:[10.1007/s001450010016](https://doi.org/10.1007/s001450010016) (<https://doi.org/10.1007%2Fs001450010016>). ISSN 0933-2790 (<https://search.worldcat.org/issn/0933-2790>).
63. Isom, Kyle. "Practical Cryptography With Go" (<https://leanpub.com/gocrypto/read#leanpub-auto-rsa>). Retrieved 4 January 2022.

Further reading

- Menezes, Alfred; van Oorschot, Paul C.; Vanstone, Scott A. (October 1996). *Handbook of Applied Cryptography* (<https://archive.org/details/handbookofapplie0000mene>). CRC Press. ISBN 978-0-8493-8523-0.
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001). *Introduction to Algorithms* (2nd ed.). MIT Press and McGraw-Hill. pp. 881 (https://archive.org/details/introductiontoal00corm_691/page/n903)–887. ISBN 978-0-262-03293-3.

External links

- The Original RSA Patent as filed with the U.S. Patent Office by Rivest; Ronald L. (Belmont, MA), Shamir; Adi (Cambridge, MA), Adleman; Leonard M. (Arlington, MA), December 14, 1977, **U.S. patent 4,405,829** (<https://patents.google.com/patent/US4405829>).
- RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2 (<https://datatracker.ietf.org/doc/html/rfc8017>)
- Explanation of RSA using colored lamps (<https://www.youtube.com/watch?v=vgTtHV04xRI>) on YouTube
- Thorough walk through of RSA (https://www.di-mgt.com.au/rsa_alg.html)
- Prime Number Hide-And-Seek: How the RSA Cipher Works (<https://www.muppetlabs.com/~breandbox/txt/rsa.html>)
- Onur Aciicmez, Cetin Kaya Koc, Jean-Pierre Seifert: *On the Power of Simple Branch Prediction Analysis* (<https://eprint.iacr.org/2006/351>)
- Interactive presentation of RSA by CrypTool (<https://www.cryptool.org/en/cto/rsa-didactic/>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=RSA_cryptosystem&oldid=1353201402"

